

```
//DATATYPES
```

```
#include<iostream>
```

```
int main(){
```

```
    int age=22;
```

```
    double mark =96.8;
```

```
    char grade = 'O';
```

```
    // string- objects that represents a sequence of character
```

```
    std::string name ="Bala Santhosh";
```

```
    bool pass=true;
```

```
    std::cout <<"Name "<<name<<" of age "<<age<<" got mark of "<<mark<<" with "<<pass;
```

```
    return 0;
```

```
}
```

```
//CONSTANT
```

```
#include <iostream>
```

```
int main()
```

```
{
```

```
    //const keyword specifies the variable to constant
```

```
    //prevents it from modifying
```

```
    //read-only
```

```
    //variable name should be in uppercase
```

```
    const double PI =3.145;
```

```
    std::cout << PI;
```

```
    return 0;
```

```
}
```

```

//NAMESPACE
#include <iostream>

namespace first{
    int x=1;
}

namespace second{
    int x=2;
}

int main()
{
    //Namespace provides a solution for preventing name conflicts in large projects.
    //Each entity needs a unique name.
    //A namespace allows for identically named entities as long as the namespaces are different.
    //with the help of namespace we can use same name for many entities without clashing with
    others by creating isolating areas

    std::cout << first::x <<'\n'; //:: local dereferencing operator
    std::cout << second::x <<'\n';

    //or
    using namespace first;
    std::cout << x <<'\n';

    //or
    using namespace std;
    cout << "hello";

    //or
    using std::cout;
    using std::string;
    string name="bala santhosh";
    cout << name;

    return 0;
}

```

```
//TYPEDEF
#include <iostream>
typedef std::string str_t;
using text_t=std::string; //we can use using keyword instead of typedef
int main()
{
    //typedef keyword provides additional name for the data types.
    //it is like a nickname for the datatype9
    str_t name = "Bala santhosh";
    std::cout << name;
    text_t name1="Bala santhosh1";
    std::cout << name1;
    return 0;
}
```

```
//TYPECAST
#include <iostream>
int main()
{
    //type conversion or typecast - conversion a value of one datatype to another datatype
    //implicit - auto
    //explicit-()
    char a = 100;
    std::cout << a;
    std::cout<< (char) 100;
    return 0;
}
```

```
//USERINPUT
#include <iostream>

//>> extraction operator
int main()
{
    std::string name;
    int age;
    std::cout<< "Enter your name ";
    std::getline(std::cin >> std::ws,name);//to get a line of string use this getline.
    //To clear the buffer new line character after using cin use std::ws to eliminate the white space
    std::cout<<"enter your age ";
    std::cin >>age;
    std::cout<<name <<" "<<age;
    return 0;
}
```

```
//MATHFUNCTION
#include <iostream>
#include<cmath>
int main()
{
    int a=2;
    int b=9;
    double d=3.14;
    int c;
    //c=std::max(a,b);
    //c=std::min(a,b);
    //c=sqrt(b);
    //c=pow(a,b);
    //c=ceil(d);
```

```
//c=floor(d);  
//c=round(d);  
//c=abs(-10);  
std::cout << c;  
return 0;  
}
```

IF and SWITCH and CONDITIONAL OPERATOR and LOGICAL are same as in C

//STRING FUNCTIONS

```
#include <iostream>  
int main()  
{  
    std::string str;  
    std::cout << "Enter your name" << '\n';  
    std::getline(std::cin >> std::ws, str);  
    std::cout << str.length();  
    std::cout << str.at(10);  
    std::cout << str.empty();  
    str.clear();  
    std::cout << str;  
    str.append("PB");  
    std::cout << str;  
    str.insert(1, "sfe");  
    std::cout << str;  
    std::cout << str.find('B');  
    str.erase(0, 2);  
    return 0;  
}
```

```
//RANDOM NUMBER
#include <iostream>
#include<ctime>
int main()
{
    srand(time(0));//we have to seed the value in srand..else it seed as srand(1).
    //in this code it will generate random number from current time as starting value
    std::cout<<rand()<<'\\n';
    return 0;
}
```

FOR LOOP ,WHILE LOOP,DO WHILE LOOP,FUNCTIONS and OVERLOADING are same as in C.

```
//ARRAYS,SIZEOF,FUNCTIONS,PASSING ARGUMENTS,RETURN TYPE,FOR EACH LOOP,PASSING ARRAY  
IN FUNCTION
```

```
#include<iostream>
```

```
std::string searchElement(int array[],int size,int element);
```

```
int main(){
```

```
    int array[5];
```

```
    std::cout << "Enter the elements of the array\n";
```

```
    for(int i=0;i<5;i++){
```

```
        std::cin >> array[i];
```

```
    }
```

```
    std::cout<<"Elements are\n";
```

```
    for(int elements:array){
```

```
        std::cout<<elements<<"\n";
```

```
    }
```

```
    std::cout<<"Enter the element to be searched\n";
```

```
    int search;
```

```
    std::cin>>search;
```

```
    std::cout<<"Element " <<searchElement(array,sizeof(array)/sizeof(array[0]),search);
```

```
    return 0;
```

```
}
```

```
std::string searchElement(int array[],int size,int element)
```

```
{
```

```
    for(int i=0;i<size;i++){
```

```
        if(array[i]==element)
```

```
            return "found";
```

```
    }
```

```
    return "Not Found";
```

```
}
```

```
//FILL FUNCTION
#include <iostream>

int main()
{
    std::string array[99];
    const int size=99;
    fill(array,array+(size/3),"1");
    fill(array+(size/3),array+(size/3)*2,"2");
    fill(array+(size/3)*2,array+size,"3");
    for(std::string arr:array){
        std::cout<<arr<<"\n";
    }
    return 0;
}
```

```
//MULTIDIMENSIONAL ARRAY

#include<iostream>

int main(){

    std::string question[3]={"1.What is array?","2.What is Pointer?","What is Structure?"};

    std::string answer[][3]={"(A) Collection of same Data","(B) Collection of different","(C) None of the above"},

    {"(A) Stores address","(B)Stores pointers","(C) None of the above"},

    {"(A)Stores same data","(B) Stores different data","(C) None"}};

    char result[]={'A','A','B'};

    for(int i=0;i<(sizeof(question)/sizeof(question[0]));i++){

        std::cout << "*****\n";

        std::cout << question[i] <<'\n';

        char res;

        for(int j=0;j<(sizeof(answer[0])/sizeof(answer[i][0]));j++)

            std::cout << answer[i][j] <<'\n';

        std::cout << "*****\n";

        std::cin>>res;

        res=toupper(res);

        if(res==result[i]){

            std::cout <<"Correct\n";

        }else{

            std::cout<< "Wrong\n";

        }

    }

}
```

//Const parameter - parameter that is effectively read-only

//code is more secure & conveys intent

//useful for references and pointers

//POINTER -It stores the address of the variable.

```
#include<iostream>
```

```
int main(){  
    int a=10;  
    int *b=&a;  
    int **c=&b;  
    int array[]={1,2,3,4,5};  
    int *d=array;  
    std::cout<<a<<'\\n';  
    std::cout<<*b<<'\\n';  
    std::cout<<*d;  
    return 0;  
}
```

//NULL POINTER – it is a special value that means something has no value. Keyword-nullptr.

```
#include <iostream>
```

```
int main()  
{  
    int * ptr =nullptr;  
    int a=10;  
    ptr=&a;  
    if(ptr==nullptr){  
        std::cout<<"No address assigned";  
    }else{  
        std::cout<<"Address assigned.Value is "<<*ptr;  
    }  
    return 0;  
}
```

//DYNAMIC MEMORY ALLOCATION – memory is allocated during runtime and memory allocated in heap

```
#include<iostream>
```

```
int main(){
```

```
    int *ptr=NULL;
```

```
    std::cout<<"Enter the size of the array\n";
```

```
    ptr=new int;
```

```
    std::cin>>(*ptr);
```

```
    std::cout<<*ptr<<'\\n';
```

```
    char *array=NULL;
```

```
    array=new char[*ptr];
```

```
    std::cout<<"Enter the elements:\n";
```

```
    for(int i=0;i<*ptr;i++){
```

```
        std::cin>>array[i];
```

```
    }
```

```
    std::cout<<"Elements are:\n";
```

```
    for(int i=0;i<*ptr;i++){
```

```
        std::cout<<array[i]<<'\\n';
```

```
    }
```

```
}
```

//FUNCTION TEMPLATE -The simple idea is to pass the data type as a parameter so that we don't need to write the same code for different data types.

```
#include <iostream>
```

```
template <typename T,typename U>
```

```
U add(T a,U b){
```

```
    return (a>b)?a:b;
```

```
}
```

```
int main(){
```

```
    std::cout<<add(2,3.1);
```

```
    return 0;
```

```
}
```

//STRUCTURE -that group related variable under one name structs that contain different data types.

//Variables inside the STRUCTURE is called as members. (.) is called as Class member access operator.

```
#include <iostream>
```

```
struct Student{
```

```
    std::string name;
```

```
    int rollNo;
```

```
};
```

```
int main()
```

```
{
```

```
    Student Student1;
```

```
    Student1.name="Bala";
```

```
    Student1.rollNo=10;
```

```
    std::cout<<"Name : "<<Student1.name<<"\n";
```

```
    std::cout<<"RollNo : "<<Student1.rollNo<<"\n";
```

```
    Student Student2;
```

```
    Student2.name="Santhosh";
```

```
    Student2.rollNo=7;
```

```
    std::cout<<"Name : "<<Student2.name<<"\n";
```

```
    std::cout<<"RollNo : "<<Student2.rollNo<<"\n";
```

```
    return 0;
```

```
}
```

```
//ENUM
```

```
#include <iostream>
```

```
int main()
```

```
{
```

```
    enum data {red=10,blue}; //user defined data type that contains the paired integer constants.
```

```
    std::cout<<blue;
```

```
    return 0;
```

```
}
```

//Objects- collections of attributes(characteristics) and methods(Functions).

//Class - it is blue print

//Overloading constructor

```
#include <iostream>
```

```
class Car{
```

```
    public:
```

```
        std::string model;
```

```
        std::string color;
```

```
        int year;
```

```
    Car(std::string model,std::string color,int year){
```

```
        this->model=model;
```

```
        this->year=year;
```

```
        this->color=color;
```

```
    }
```

```
    void displayDetails(){
```

```
        std::cout<<model<<"\n";
```

```
        std::cout<<color<<"\n";
```

```
        std::cout<<year<<"\n";
```

```
    }
```

```
    void changeColor(std::string color){
```

```
        this->color=color;
```

```
    }
```

```
};
```

```
int main()
```

```
{
```

```
    Car myCar("Alto","Blue",2014);
```

```
    myCar.displayDetails();
```

```
    myCar.changeColor("Black");
```

```
    myCar.displayDetails();
```

```
    return 0;
```

```
}
```

```
//GETTERS AND SETTERS – to get and set the private members
```

```
//#include<iostream>
```

```
class Rank{
```

```
    private:
```

```
        int rank;
```

```
    public:
```

```
        Rank(){
```

```
        }
```

```
        Rank(int rank){
```

```
            setRank(rank);
```

```
        }
```

```
        void setRank(int rank){
```

```
            this->rank=rank;
```

```
        }
```

```
        int getRank(){
```

```
            return rank;
```

```
        }
```

```
};
```

```
int main(){
```

```
    Rank rank1;
```

```
    rank1.setRank(10);
```

```
    std::cout<<rank1.getRank();
```

```
    return 0;
```

```
}
```

//INHERITANCE - if one class acquires the properties and characteristics of other class.

```
#include<iostream>
```

```
class Animal{
```

```
    public:
```

```
        bool alive=true;
```

```
    void eat(){
```

```
        std::cout<<"Animal is Eating\n";
```

```
    }
```

```
};
```

```
class Dog: public Animal{
```

```
    public:
```

```
    void bark(){
```

```
        std::cout<<"Barking\n";
```

```
    }
```

```
};
```

```
class Cat: public Animal{
```

```
    public:
```

```
    void meow(){
```

```
        std::cout<<"Meowinggg\n";
```

```
    }
```

```
};
```

```
int main(){
```

```
    Cat cat;
```

```
    cat.eat();
```

```
    cat.meow();
```

```
    Dog dog;
```

```
    dog.eat();
```

```
    dog.bark();
```

```
}
```